

EXPERIMENT NO. 11

Task 1: Multithreaded Counter using POSIX Threads Demonstrating Race Condition

Creates multiple concurrent threads updating a shared variable without synchronization, exposing race conditions.

Complete code:

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

#define NUM_THREADS 5
#define ITERATIONS_PER_THREAD 200000

volatile long shared_counter = 0;

void *worker_thread(void *arg) {
    long id = (long)arg;
    for (int i = 0; i < ITERATIONS_PER_THREAD; i++) {
        /* Unprotected critical section: causes race condition */
        shared_counter++;
    }
    pthread_exit(NULL);
}

int main() {
    pthread_t threads[NUM_THREADS];
    long expected_total = (long)NUM_THREADS * ITERATIONS_PER_THREAD;

    printf("===== MULTITHREADED COUNTER: RACE CONDITION =====\n");
    printf("Spawning %d POSIX threads, each incrementing counter %d times...\n",
        NUM_THREADS, ITERATIONS_PER_THREAD);
    printf("Expected Final Total: %ld\n", expected_total);

    for (long i = 0; i < NUM_THREADS; i++) {
        if (pthread_create(&threads[i], NULL, worker_thread, (void *)i) != 0) {
            perror("pthread_create failed");
            exit(1);
        }
    }

    for (int i = 0; i < NUM_THREADS; i++) {
        pthread_join(threads[i], NULL);
    }

    printf("Actual Final Counter: %ld\n", shared_counter);
    printf("Lost Updates: %ld (%.2f%% loss due to unsynchronized race condition)\n",
        expected_total - shared_counter,
        ((double)(expected_total - shared_counter) / expected_total) * 100.0);
    printf("Result: Multiple threads interleaved read-modify-write instructions concurrently.\n");
    return 0;
}
```

Output screenshot:

```
teja27@TEJA:~$ nano exp11_task1.c
teja27@TEJA:~$ gcc -pthread exp11_task1.c -o exp11_task1
teja27@TEJA:~$ ./exp11_task1
===== MULTITHREADED COUNTER: RACE CONDITION =====
Spawning 5 POSIX threads, each incrementing counter 200000 times...
Expected Final Total: 1000000
Actual Final Counter: 353548
Lost Updates: 646452 (64.65% loss due to unsynchronized race condition)
Result: Multiple threads interleaved read-modify-write instructions concurrently.
teja27@TEJA:~$
```

Task 2: Synchronized Counter using pthread_mutex_lock() and pthread_mutex_unlock()

Enforces mutual exclusion with POSIX mutex locks to eliminate lost updates and ensure deterministic results.

Complete code:

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

#define NUM_THREADS 5
#define ITERATIONS_PER_THREAD 200000

volatile long shared_counter = 0;
pthread_mutex_t counter_mutex;

void *synchronized_worker(void *arg) {
    long id = (long)arg;
    for (int i = 0; i < ITERATIONS_PER_THREAD; i++) {
        /* Acquire mutex lock before entering critical section */
        pthread_mutex_lock(&counter_mutex);
        shared_counter++;
        /* Release mutex lock upon leaving critical section */
        pthread_mutex_unlock(&counter_mutex);
    }
    pthread_exit(NULL);
}

int main() {
    pthread_t threads[NUM_THREADS];
    long expected_total = (long)NUM_THREADS * ITERATIONS_PER_THREAD;

    /* Initialize POSIX mutex */
    if (pthread_mutex_init(&counter_mutex, NULL) != 0) {
        perror("pthread_mutex_init failed");
        return 1;
    }

    printf("===== MUTEX SYNCHRONIZED COUNTER (PTHREAD_MUTEX) =====\n");
    printf("Spawning %d POSIX threads with pthread_mutex protection...\n", NUM_THREADS);
    printf("Expected Final Total: %ld\n", expected_total);

    for (long i = 0; i < NUM_THREADS; i++) {
        if (pthread_create(&threads[i], NULL, synchronized_worker, (void *)i) != 0) {
            perror("pthread_create failed");
            exit(1);
        }
    }

    for (int i = 0; i < NUM_THREADS; i++) {
        pthread_join(threads[i], NULL);
    }

    pthread_mutex_destroy(&counter_mutex);

    printf("Actual Final Counter: %ld\n", shared_counter);
    printf("Lost Updates: %ld (0.00% loss - 100% Accurate Execution!)\n",
        expected_total - shared_counter);
    printf("Comparison Analysis:\n"
        "1. Race condition version allowed overlapping increments, discarding writes.\n"
        "2. Mutex version guarantees mutual exclusion: only one thread executes the\n"
        "   critical section at any instance, guaranteeing total data consistency.\n");
    return 0;
}
```

Output screenshot:

```
teja27@TEJA:~$ nano exp11_task2.c
teja27@TEJA:~$ gcc -pthread exp11_task2.c -o exp11_task2
teja27@TEJA:~$ ./exp11_task2
===== MUTEX SYNCHRONIZED COUNTER (PTHREAD_MUTEX) =====
Spawning 5 POSIX threads with pthread_mutex protection...
Expected Final Total: 1000000
Actual Final Counter: 1000000
Lost Updates: 0 (0.00% loss - 100% Accurate Execution!)
Comparison Analysis:
1. Race condition version allowed overlapping increments, discarding writes.
2. Mutex version guarantees mutual exclusion: only one thread executes the
   critical section at any instance, guaranteeing total data consistency.
teja27@TEJA:~$ █
```